

Funciones

Práctica 3

Funciones

- ▶ Una **función** es una serie de instrucciones que le dicen a la máquina qué hacer con las entradas, para así obtener una salida
- ▶ Crear funciones es fundamental porque permite reutilizar código, mejorar la legibilidad y facilitar el mantenimiento de tus programas. Además, se evita repetir instrucciones y centralizar los cambios

Funciones - sintaxis

En Python para definir una función se utiliza el comando **def**, cada función está compuesta por su nombre (nombre_funcion) y objetos de entrada o argumentos.

```
def nombre_funcion(argumentos):  
    # Bloque de código con indentación  
    expresión1  
    expresión2  
    return valor_opcional
```

- ▶ **Nombre:** debe describir la tarea que realiza
- ▶ **Argumentos:** datos que la función recibe para trabajar
- ▶ **return:** indica los objetos de salida de la función. El valor de retorno puede ser un número, una lista, un gráfico, un mensaje...

Importante:

Todo el código dentro de la función debe estar indentado

Las variables o cálculos que se encuentran dentro de la función solo existen internamente adentro de ella, por eso se las denomina variables locales.

Funciones - sintaxis

Ejemplo

```
def func_cuad(x, a2, a1, a0):  
    x2=x**2  
    x_out = a2 * x2 + a1 * x + a0  
    return x_out
```

Para aplicar la función:

```
## defino una variable previa  
valor_x = 5
```

```
#aplico la funcion  
imagen = func_cuad(x= valor_x, a2=3, a1=0, a0=5)
```

```
## otra forma  
#Si no indico el nombre del argumento, lo ingresado lo interpret  
#en el mismo orden de los argumentos.  
imagen = func_cuad(valor_x, 3, 0, 5)
```

Funciones - argumentos por default

Valor por default

Los valores por default son un valor preestablecido que si no se ingresa, toma dicho valor. Siempre **deben ubicarse al final de la función**

```
def func_cuad(x, a2, a1, a0=0):
```

```
    x2=x**2
```

```
    x_out = a2 * x2 + a1 * x + a0
```

```
    return x_out
```

Funciones - Buenas prácticas

Es recomendable que las funciones (y es una buena práctica) que sean autodescriptas, es decir, que dentro de sí misma explique en qué consiste, de qué tipo y que representen los argumentos de entrada. Al igual que como cuando buscan funciones y encuentran la documentación asociada. Para ello es útil el uso de las cadenas de caracteres con triple comilla.

```
def func_cuad(x, a2, a1, a0=0):  
    '''  
    Descripción de la función  
    y de los argumentos de la variables  
    x = float, constante, valor de dominio de la función  
    a2 = float, constante, coeficiente del término cuadrático  
    a1 = float, constante, coeficiente del término lineal  
    a0 = float, constante, ordenada al origen, Default: 0  
    '''  
  
    x2=x**2  
    x_out = a2 * x2 + a1 * x + a0  
    return x_out
```

Funciones - Salida y extracción múltiple

Supongamos que tenemos una función **func** que tiene ciertos argumentos de entrada, y dentro de la misma se generan 3 variables locales que son las salidas y1, y2 e y3

```
def func(x1, x2, x3):  
    operaciones  
    return y1, y2, y3
```

y1,y2,y3 son una tupla, entonces puedo pensarla como un solo objeto de salida.

Si quiero aplicar la funcion puedo usar:

```
output= func(i1,i2,i3)
```

o tambien

```
out1, out2, out3 = func(i1, i2, i3)
```

En este último ejemplo la salida son 3 variables distintas y se asignan separadas por , (coma). Esto desempaqueta la tupla.

Funciones - Salida y extracción múltiple

Ejemplo

```
def calcular_estadisticas(numeros):  
    """Calcula el valor mínimo, máximo y el promedio  
    de una lista de números"""  
    minimo = min(numeros)  
    maximo = max(numeros)  
    promedio = sum(numeros) / len(numeros)  
    return minimo, maximo, promedio
```

Aplicando la funcion

```
notas = [7, 9, 4, 10, 6, 8]  
  
estadisticos = calcular_estadisticas(notas)  
  
## o tambien puedo asignar cada una por separado  
nota_min, nota_max, nota_prom = calcular_estadisticas(notas)
```

Funciones lambda

- ▶ Los **funciones lambda** son funciones anónimas y de una sola línea, devolviendo como valor el resultado de la expresión definida en ella. Son versiones más concisas de las funciones convencionales

Funciones lambda - sintaxis

`lambda` argumentos: expresión

- ▶ **lambda**: es el comando para indicar que es una función de este tipo
- ▶ **Argumentos**: son los inputs de la función que puede ser ninguno, uno o más.
- ▶ **expresión**: es la operación que se quiere ejecutar.

Ejemplo: Asigno a una variable la función lambda, y con esto podría reutilizarla

```
restar = lambda x, y: x - y  
print( restar(10, 8) )
```

Funciones lambda

Las funciones lambda se utilizan a menudo en combinación con funciones de orden superior, es decir, funciones que toman otras funciones como argumentos. Los casos más comunes:

- **Map**(function, iterables): Aplica una función a cada elemento de un objeto iterable (como una lista) y devuelve un nuevo iterable con los resultados.

```
numeros = [1, 2, 3, 4, 5 ,6]
# Eleva al cuadrado cada elemento de la lista numeros
cuadrados = list(map(lambda x: x**2, numeros))

print(cuadrados) # [1, 4, 9, 16, 25, 36]
```

- **filter**(function, iterable): Crea un nuevo objeto iterable con los elementos de un iterable que cumplen con una condición

```
numeros = [1, 2, 3, 4, 5 ,6]
# Filtra los numeros pares
pares = list(filter(lambda x: x % 2 == 0, numeros))
print(pares) # [2, 4, 6]
```

Funciones lambda

- **sorted**(iterable, key, reverse): Se usa para ordenar objetos iterables. La función lambda se usa como clave (key) para ordenarlos (es decir, en base a que). El argumento reverse es opcional y es un valor booleano que si es True, los datos se ordenan de mayor a menor. Además, la función sorted ya crea una lista.

```
puntos = [(1, 5), (3, 2), (2, 8)]  
# Ordenar según el segundo elemento de cada tupla  
puntos_ordenados = sorted(puntos, key=lambda p: p[1])  
print(puntos_ordenados) # [(3, 2), (1, 5), (2, 8)]
```

Ejercicios

- 1) Se tiene un sensor que mide la concentración de partículas finas (PM2.5) en microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$). Para ello se requieren tres funciones que realicen lo siguiente:

a) **Función calcular_promedio_pm25:**

Propósito: Calcular el promedio de una lista de lecturas de PM2.5.

Parámetros: Acepta un único parámetro, `lecturas_pm25`, que es una lista de números (flotantes o enteros).

Comportamiento: La función debe sumar todos los valores de la lista y dividirlos por la cantidad de elementos.

b) **Función evaluar_calidad_aire:**

Propósito: Evaluar el nivel de calidad del aire basado en el promedio de PM2.5.

Ejercicios

Parámetros: Acepta un único parámetro, promedio, que es el valor numérico del promedio de PM2.5.

Comportamiento: Debe determinar si la calidad del aire es “Buena”, “Aceptable” o “Mala” según los siguientes criterios y debe indicar si se necesita una alerta.

Buena: Si el promedio es menor o igual a 15

Aceptable: $15 < \text{promedio} < 30$

Mala: promedio mayor o igual a 30

Alerta: se emite una alerta si la calidad del aire es “Mala” o si el promedio supera los 20 $\mu\text{g}/\text{m}^3$.

c) **Función imprimir_reporte:**

Propósito: Imprimir un reporte final

Parámetros: Acepta un parámetro: lecturas_pm25.

Comportamiento: La función debe imprimir un mensaje que incluya el promedio de PM2.5, el nivel de calidad del aire y si se necesita una alerta.

Ejercicios

Para probar el funcionamiento:

```
lecturas_diarias = [12.5, 14.2, 16.8, 18.0, 21.5, 25.1]
```

2) A partir de la lista = [1,3,5,3,2,7,4,5]

Elevarla al cuadrado utilizando:

a) Map y una función definida por ustedes

b) Map() y una función Lambda

A partir de la lista elevada al cuadrado:

1. Ordenar la lista en base a la cantidad de veces que aparece repetida
2. Obtener la lista con solo aquellos elementos mayores a 10.